

Automated Server Log Monitoring & Report Generation System

Case Overview

A company runs multiple application servers, each generating daily log files. The DevOps team needs an automated shell script that monitors server health, extracts key information, and produces a daily report — all without manual intervention.

Input: Log File Format

Log files are stored in /logs, one per server per day. Each line follows this fixed format:

```
YYYY-MM-DD HH:MM:SS LEVEL Message

2025-06-10 09:00:01 INFO User Ravi logged in
2025-06-10 09:02:15 ERROR Database connection failed
2025-06-10 09:10:20 WARNING High memory usage
```

Date	Time	Level	Message
2025-06-10	09:00:01	INFO	User Ravi logged in
2025-06-10	09:02:15	ERROR	Database connection failed
2025-06-10	09:10:20	WARNING	High memory usage
2025-06-10	09:15:33	INFO	File uploaded successfully

```
/logs/server1.log      /logs/server2.log      /logs/server3.log
/reports/daily_report_YYYY-MM-DD.txt
/script_logs/script_execution.log
```

Task 01 Environment Setup

Define all paths and settings as variables at the top of your script. This keeps the script easy to configure.

LOG_DIR	Directory containing .log files
REPORT_DIR	Directory to save generated reports
SCRIPT_LOG	Path to the script's own execution log
DATE	Today's date — used in filenames
REPORT_FILE	Full path for today's report file

Your Code — Write Below

```
#!/bin/bash

LOG_DIR=
REPORT_DIR=
SCRIPT_LOG=
DATE=$(date +%Y-%m-%d)
REPORT_FILE=$REPORT_DIR/report_$DATE.txt
```

Task 02 Input File Validation

Before processing, verify the log directory exists, contains .log files, and that files are readable. Log an error and stop if any check fails.

Your Code — Write Below

```
check_directory() {
    # Check if LOG_DIR exists

    # Check if .log files are present

    # Log error and exit if either check fails
}
```

Task 03 Log File Processing — Nested Loops

Use a for loop to iterate over all .log files. Inside each iteration, use a while read loop to process every line.

Your Code — Write Below

```
for log_file in $LOG_DIR/*.log; do

    while IFS= read -r line; do
        # Your line processing here
    done < "$log_file"

done
```

Task 04 Extract Information Using AWK

Use awk to count INFO, WARNING, and ERROR entries in each log file. Field \$3 holds the log level.

Your Code — Write Below

```
# Count each log level
info_count=$( ... "$log_file")
warn_count=$( ... "$log_file")
error_count=$( ... "$log_file")
```

Task 05 Data Cleaning Using SED

Use sed to clean each log line before analysis. Handle common issues like extra spaces, tabs, and stray special characters.

Your Code — Write Below

```
clean_line() {
    local line="$1"
    # Remove extra whitespace
```

```
# Replace tab characters

echo "$line"

}
```

Task 06 Error Detection Logic

Check health status to each server based on its error and warning counts.

ERROR > 10	Status = CRITICAL
WARNING > 20	Status = WARNING
Otherwise	Status = NORMAL

Task 07 Generate Daily Report

After processing all files, write a formatted summary report. Use echo and >> to append content to REPORT_FILE.

Expected output:

Server	INFO	WARNING	ERROR	STATUS
server1	120	5	1	NORMAL
server2	80	12	15	CRITICAL
server3	95	3	0	NORMAL

Your Code — Write Below

```
generate_report() {
    echo "Daily Server Health Report" >> "$REPORT_FILE"
    echo "Generated: $(date)" >> "$REPORT_FILE"
    # Write one row per server
}
```

Task 08 Maintain Script Execution Log

Record each key action to SCRIPT_LOG with a timestamp and level label so runs are traceable.

```
[2025-06-10 23:00:00] [INFO]    Script started
[2025-06-10 23:00:03] [WARNING] High error count in server2
[2025-06-10 23:00:05] [INFO]    Script completed
```

Your Code — Write Below

```
write_log() {
    local level="$1"
    local msg="$2"
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] [$level] $msg" >> "$SCRIPT_LOG"
}

# Usage: write_log "INFO" "Script started"
```

Task 09 Implement Functions

Refactor all logic into named functions. This keeps the script modular and easy to maintain.

check_directory	Validate the log directory and its files
process_log_file	Analyse one file and determine server status
generate_report	Write the final formatted report
write_log	Append a timestamped entry to SCRIPT_LOG
show_menu	Display the interactive menu (Task 10)

Your Code — Write Below

```
check_directory() { ... }
process_log_file() { ... }
generate_report() { ... }
write_log()      { ... }
show_menu()      { ... }

# Main
write_log "INFO" "Script started"
check_directory
for f in $LOG_DIR/*.log; do process_log_file "$f"; done
generate_report
```

Task 10 Menu System Using Case Statement

Provide an interactive menu so operators can manually trigger specific actions. Use a case statement to route each selection.

```
1. Analyze logs      2. Generate report
3. View report      4. Exit
```

Your Code — Write Below

```
show_menu() {
    echo "1. Analyze logs"
    echo "2. Generate report"
    echo "3. View report"
    echo "4. Exit"
    read -p "Select [1-4]: " choice
    case $choice in
        1) ... ;; 2) ... ;; 3) ... ;; 4) exit 0 ;; *) echo "Invalid" ;;
    esac
}
```

Task 11 Cron Job Scheduling

Schedule the script to run automatically every night at 11:00 PM using crontab -e.

MIN	0 (at minute 0)
HOURL	23 (11 PM, 24-hour clock)
DAY	* (every day)

MONTH	* (every month)
WEEKDAY	* (every day of week)

Your Code — Write Below

Run: `crontab -e` then add:

```
0 23 * * * /scripts/log_monitor.sh >> /script_logs/cron.log 2>&1
```

Verify with: `crontab -l`